



Python for Data Science - 2305CS303

Lab - 11

Roll No. : 122

Name : Yatharth Sanghavi

```
import pandas as pd
```

1. Create a MultiIndexed DataFrame

Create a DataFrame containing student marks for different Departments and Semesters. Use `set_index()` to create a MultiIndex using `Department` and `Semester`. Display the DataFrame and its index.

```
student_data={'rollno':[1,2,3,4],
              'name':['yatharth','sahil','vimal','akshay'],
              'department':['CE','MECH','CE','MECH'],
              'semester':[2,3,4,5],
              'marks':[12,34,56,78],
              'city':['rajkot','jamnagar','rajkot','dhandhuka']}
student = pd.DataFrame(student_data)
student.set_index(['department','semester'])
```

		rollno	name	marks	city
department	semester				
CE	2	1	yatharth	12	rajkot
MECH	3	2	sahil	34	jamnagar
CE	4	3	vimal	56	rajkot
MECH	5	4	akshay	78	dhandhuka

2. Create a MultiIndex using Multiple Columns

Create a DataFrame containing `Department`, `Year`, `Gender`, `Student` and `Marks`. Set `Department`, `Year` and `Gender` as the MultiIndex and display the resulting DataFrame.

```
student_data={'rollno':[1,2,3,4],
              'name':['yatharth','sahil','vimal','akshay'],
              'department':['CE','MECH','CE','MECH'],
              'semester':[2,3,4,5],
              'marks':[12,34,56,78],
              'year':[20,21,22,23],
              'gender':['Male','Male','Male','Male'],
              'city':['rajkot','jamnagar','rajkot','dhandhuka']}
student = pd.DataFrame(student_data)
student.set_index(['department','year','gender'])
```

			rollno	name	semester	marks	city
department	year	gender					
CE	20	Male	1	yatharth	2	12	rajkot
MECH	21	Male	2	sahil	3	34	jamnagar
CE	22	Male	3	vimal	4	56	rajkot
MECH	23	Male	4	akshay	5	78	dhandhuka

3. Read a MultiIndexed DataFrame Directly from CSV

Create a CSV file containing columns `Department`, `Semester`, `Subject`, and `Marks`. Read the CSV file directly as a MultiIndexed DataFrame using `pd.read_csv()` with the appropriate `index_col` parameter. Display the resulting DataFrame.

```
csv_data = """Department,Semester,Subject,Marks
CE,2,Python,85
CE,2,Maths,90
MECH,3,Thermodynamics,78
MECH,3,Fluid Mechanics,82
"""

with open("student_marks.csv", "w") as f:
    f.write(csv_data)

df_csv = pd.read_csv("student_marks.csv", index_col=["Department", "Semester"])
print("MultiIndexed DataFrame from CSV:")
display(df_csv)
```

MultiIndexed DataFrame from CSV:

		Subject	Marks
Department	Semester		
CE	2	Python	85
	2	Maths	90
MECH	3	Thermodynamics	78
	3	Fluid Mechanics	82

4. Inspect a MultiIndexed DataFrame

Using the MultiIndexed DataFrame created in Question 3, display the index levels, index names, columns, shape, and data types.

```
print("Index Levels:", df_csv.index.levels)
print("\nIndex Names:", df_csv.index.names)
print("\nColumns:", df_csv.columns.tolist())
print("\nShape:", df_csv.shape)
print("\nData Types:\n", df_csv.dtypes)
```

```
Index Levels: [['CE', 'MECH'], [2, 3]]
Index Names: ['Department', 'Semester']
Columns: ['Subject', 'Marks']
Shape: (4, 2)
Data Types:
Subject    object
Marks      int64
dtype: object
```

5. Access Data using MultiIndex

Using a MultiIndexed DataFrame having `Department` and `Semester` as indexes, access all records belonging to one department and one specific department-semester combination.

```
print("--- Records for 'CE' Department ---")
print(df_csv.loc["CE"])

print("\n--- Records for 'CE', Semester 2 ---")
print(df_csv.loc[("CE", 2)])
```

```

--- Records for 'CE' Department ---
      Subject  Marks
Semester
2          Python    85
2           Maths    90

--- Records for 'CE', Semester 2 ---
      Subject  Marks
Department Semester
CE          2      Python    85
CE          2          Maths    90

```

6. Cross Section using xs()

Create a MultIndexed DataFrame with `Department` and `Semester` as index levels. Use `xs()` to retrieve the records for a particular semester across all departments.

```

print("--- Cross-section for Semester 2 across all departments ---")
print(df_csv.xs(key=2, level="Semester"))

```

```

--- Cross-section for Semester 2 across all departments ---
      Subject  Marks
Department
CE          Python    85
CE           Maths    90

```

7. Cross Section from a Specific Level

Using the same MultIndexed DataFrame, use `xs()` with the `level` parameter to retrieve records for one department. Display the result.

```

print("--- Cross-section for 'CE' Department ---")
print(df_csv.xs(key="CE", level="Department"))

```

```

--- Cross-section for 'CE' Department ---
      Subject  Marks
Semester
2          Python    85
2           Maths    90

```

8. Cross Section on Multiple Index Levels

Create a MultIndexed DataFrame with `Department`, `Semester`, and `Subject` as index levels. Use `xs()` to retrieve the records corresponding to a particular combination of Department and Semester.

```

df_3level = pd.read_csv(
    "student_marks.csv", index_col=["Department", "Semester", "Subject"]
)
print(df_3level.xs(key=("CE", 2), level=["Department", "Semester"]))

```

```

      Marks
Subject
Python    85
Maths     90

```

9. Identify Missing Data

Create a student DataFrame containing `Name`, `Department`, `Marks`, `Attendance`, and `City`, with some missing values represented by `NaN` or `None`. Display the DataFrame and identify the missing values using `isna()`.

```

import numpy as np
data_missing = {
    "Name": ["Yatharth", "Sahil", "Vimal", "Akshay"],
    "Department": ["CE", "MECH", "CE", None],
    "Marks": [85, np.nan, 90, 78],
    "Attendance": [92, 80, np.nan, 88],
    "City": ["Rajkot", "Jamnagar", None, "Dhandhuka"],
}
df_student = pd.DataFrame(data_missing)

print("Original DataFrame:")
display(df_student)

```

```
print("\nMissing values identification using isna():")
display(df_student.isna())
```

Original DataFrame:

	Name	Department	Marks	Attendance	City
0	Yatharth	CE	85.0	92.0	Rajkot
1	Sahil	MECH	NaN	80.0	Jamnagar
2	Vimal	CE	90.0	NaN	None
3	Akshay	None	78.0	88.0	Dhandhuka

Missing values identification using isna():

	Name	Department	Marks	Attendance	City
0	False	False	False	False	False
1	False	False	True	False	False
2	False	False	False	True	True
3	False	True	False	False	False

10. Count Missing Values

Using the DataFrame from Question 9, calculate the number of missing values in each column and display the total number of missing values in the complete DataFrame.

```
print("Missing values count per column:")
print(df_student.isna().sum())

print(
    "\nTotal missing values in complete DataFrame:", df_student.isna().sum().sum()
)
```

Missing values count per column:

```
Name      0
Department 1
Marks      1
Attendance 1
City       1
dtype: int64
```

Total missing values in complete DataFrame: 4

11. Detect Missing Values using isnull()

Create a DataFrame containing missing values in different columns. Use `isnull()` and `notnull()` to display the missing and non-missing values. Compare the results of `isnull()` and `isna()`.

```
print("Using isnull():")
display(df_student.isnull())

print("\nUsing notnull():")
display(df_student.notnull())

are_identical = df_student.isnull().equals(df_student.isna())
print(
    f"\nAre isnull() and isna() identical? {are_identical} (isna is an alias for isnull)"
)
```

Using isnull():

	Name	Department	Marks	Attendance	City
0	False	False	False	False	False
1	False	False	True	False	False
2	False	False	False	True	True
3	False	True	False	False	False

Using notnull():

	Name	Department	Marks	Attendance	City
0	True	True	True	True	True
1	True	True	False	True	True
2	True	True	True	False	False
3	True	False	True	True	True

Are isnull() and isna() identical? True (isna is an alias for isnull)

12. Remove Missing Values

Using a DataFrame containing missing values, demonstrate `dropna()` to remove rows containing at least one missing value. Also demonstrate how to remove columns containing missing values.

```
print("--- DataFrame after dropping rows with any missing value ---")
display(df_student.dropna(axis=0))

print("\n--- DataFrame after dropping columns with any missing value ---")
display(df_student.dropna(axis=1))
```

--- DataFrame after dropping rows with any missing value ---

	Name	Department	Marks	Attendance	City
0	Yatharth	CE	85.0	92.0	Rajkot

--- DataFrame after dropping columns with any missing value ---

	Name
0	Yatharth
1	Sahil
2	Vimal
3	Akshay

13. Fill Missing Values with a Constant

Create a DataFrame containing missing values in the `Marks` and `City` columns. Replace missing values using suitable constant values with `fillna()`.

```
df_filled_const = df_student.fillna({"Marks": 0, "City": "Unknown"})

print("DataFrame after filling missing values with constants:")
display(df_filled_const)
```

DataFrame after filling missing values with constants:

	Name	Department	Marks	Attendance	City
0	Yatharth	CE	85.0	92.0	Rajkot
1	Sahil	MECH	0.0	80.0	Jamnagar
2	Vimal	CE	90.0	NaN	Unknown
3	Akshay	None	78.0	88.0	Dhandhuka

14. Fill Missing Values using Mean and Median

Create a DataFrame containing missing numerical values. Fill missing values in the `Marks` column using the mean and demonstrate filling another numerical column using the median.

```
df_num_filled = df_student.copy()

mean_marks = df_num_filled["Marks"].mean()
df_num_filled["Marks"] = df_num_filled["Marks"].fillna(mean_marks)

median_att = df_num_filled["Attendance"].median()
df_num_filled["Attendance"] = df_num_filled["Attendance"].fillna(median_att)

print(
    f"Mean Marks used: {mean_marks:.2f} | Median Attendance used: {median_att:.2f}"
)
display(df_num_filled)
```

Mean Marks used: 84.33 | Median Attendance used: 88.00

	Name	Department	Marks	Attendance	City
0	Yatharth	CE	85.000000	92.0	Rajkot
1	Sahil	MECH	84.333333	80.0	Jamnagar
2	Vimal	CE	90.000000	88.0	None
3	Akshay	None	78.000000	88.0	Dhandhuka

15. Fill Missing Categorical Values using Mode

Create a DataFrame containing missing values in a categorical column such as `City` or `Department`. Fill the missing values using the mode of that column.

```
df_cat_filled = df_student.copy()

dept_mode = df_cat_filled["Department"].mode()[0]
df_cat_filled["Department"] = df_cat_filled["Department"].fillna(dept_mode)

city_mode = df_cat_filled["City"].mode()[0]
df_cat_filled["City"] = df_cat_filled["City"].fillna(city_mode)

print(f"Department Mode: {dept_mode} | City Mode: {city_mode}")
display(df_cat_filled)
```

Department Mode: CE | City Mode: Dhandhuka

	Name	Department	Marks	Attendance	City
0	Yatharth	CE	85.0	92.0	Rajkot
1	Sahil	MECH	NaN	80.0	Jamnagar
2	Vimal	CE	90.0	NaN	Dhandhuka
3	Akshay	CE	78.0	88.0	Dhandhuka

16. Forward Fill and Backward Fill

Create a time-series-like DataFrame containing missing values. Demonstrate forward filling using `ffill()` and backward filling using `bfill()`. Observe the difference between the two methods.

```
ts_data = {
    "Date": pd.date_range(start="2024-01-01", periods=5),
    "Reading": [10.0, np.nan, np.nan, 25.0, np.nan],
}
df_ts = pd.DataFrame(ts_data)

print("\nForward Fill (ffill()):")
display(df_ts.fffll())

print("\nBackward Fill (bfill()):")
display(df_ts.bfill())
```

Forward Fill (ffill()):

	Date	Reading
0	2024-01-01	10.0
1	2024-01-02	10.0
2	2024-01-03	10.0
3	2024-01-04	25.0
4	2024-01-05	25.0

Backward Fill (bfill()):

	Date	Reading
0	2024-01-01	10.0
1	2024-01-02	25.0
2	2024-01-03	25.0
3	2024-01-04	25.0
4	2024-01-05	NaN

17. Groupby - Basic Grouping

Create a DataFrame containing `Department`, `Student`, and `Marks`. Group the data by `Department` and calculate the average marks for each department.

```
df_grp = pd.DataFrame(
    {
        "Department": ["CE", "MECH", "CE", "MECH", "CE"],
        "Student": ["Yatharth", "Sahil", "Vimal", "Akshay", "Rohan"],
        "Marks": [85, 78, 90, 82, 88],
    }
)

avg_marks = df_grp.groupby("Department")["Marks"].mean()
print("Average Marks by Department:")
print(avg_marks)
```

```
Average Marks by Department:
Department
CE      87.666667
MECH    80.000000
Name: Marks, dtype: float64
```

18. Groupby with Multiple Aggregations

Using a student DataFrame, group the records by `Department` and calculate the minimum, maximum, mean, and count of `Marks` for each department.

```
agg_results = df_grp.groupby("Department")["Marks"].agg(
    ["min", "max", "mean", "count"]
)
print("Multiple Aggregations on Marks by Department:")
display(agg_results)
```

```
Multiple Aggregations on Marks by Department:
```

	min	max	mean	count
Department				
CE	85	90	87.666667	3
MECH	78	82	80.000000	2

19. Groupby with Multiple Columns

Create a DataFrame containing `Department`, `Semester`, `Student`, and `Marks`. Group the data by both `Department` and `Semester` and calculate the average marks.

```
df_multi_grp = pd.DataFrame(
    {
        "Department": ["CE", "CE", "MECH", "MECH", "CE"],
```

```

        "Semester": [2, 2, 3, 3, 4],
        "Student": ["Yatharth", "Aman", "Sahil", "Akshay", "Vimal"],
        "Marks": [85, 75, 78, 82, 90],
    }
)

avg_by_dept_sem = df_multi_grp.groupby(["Department", "Semester"])[
    "Marks"
].mean()
print("Average Marks grouped by Department and Semester:")
print(avg_by_dept_sem)

```

Average Marks grouped by Department and Semester:

Department	Semester	Marks
CE	2	80.0
	4	90.0
MECH	3	80.0

Name: Marks, dtype: float64

20. Named Aggregation using agg()

Using a student DataFrame, group the data by `Department` and use `agg()` with named aggregation to calculate `Maximum_Marks`, `Minimum_Marks`, and `Average_Marks`.

```

named_agg = df_grp.groupby("Department").agg(
    Maximum_Marks=("Marks", "max"),
    Minimum_Marks=("Marks", "min"),
    Average_Marks=("Marks", "mean"),
)

print("Named Aggregation Results:")
display(named_agg)

```

Named Aggregation Results:

	Maximum_Marks	Minimum_Marks	Average_Marks
Department			
CE	90	85	87.666667
MECH	82	78	80.000000

Next steps: [Generate code with named_agg](#)

[New interactive sheet](#)

21. Groupby and Count Records

Create a DataFrame containing student details and departments. Use `groupby()` to count the number of students in each department. Also calculate the number of unique cities for each department.

```

df_details = pd.DataFrame(
    {
        "Student": ["Yatharth", "Sahil", "Vimal", "Akshay", "Rohan"],
        "Department": ["CE", "MECH", "CE", "MECH", "CE"],
        "City": ["Rajkot", "Jamnagar", "Rajkot", "Dhandhuka", "Surat"],
    }
)

dept_summary = df_details.groupby("Department").agg(
    Student_Count=("Student", "count"), Unique_Cities=("City", "nunique")
)

print("Department summary - Student Count and Unique Cities:")
display(dept_summary)

```

Department summary - Student Count and Unique Cities:

	Student_Count	Unique_Cities
Department		
CE	3	2
MECH	2	2

Next steps: [Generate code with dept_summary](#)

[New interactive sheet](#)

22. Groupby after Handling Missing Data

Create a DataFrame containing `Department`, `Marks`, and `City` with missing values. Handle the missing marks appropriately and then group the data by department to calculate average marks.

```
df_missing_grp = pd.DataFrame(
    {
        "Department": ["CE", "MECH", "CE", "MECH", "CE"],
        "Marks": [85, np.nan, 90, 82, np.nan],
        "City": ["Rajkot", "Jamnagar", None, "Dhandhuka", "Rajkot"],
    }
)

print("Original Data with missing values:")
display(df_missing_grp)

overall_mean = df_missing_grp["Marks"].mean()
df_missing_grp["Marks"] = df_missing_grp["Marks"].fillna(overall_mean)

dept_avg = df_missing_grp.groupby("Department")["Marks"].mean()

print("\nAverage Marks by Department (after replacing missing values):")
print(dept_avg)
```

Original Data with missing values:

	Department	Marks	City	
0	CE	85.0	Rajkot	
1	MECH	NaN	Jamnagar	
2	CE	90.0	None	